



PORTFOLIO

권인 포트폴리오



확장성 있는 사용자 경험을
설계하는 개발자
권인입니다.

☎ 010-7999-5534 ✉ in24041210@gmail.com

🌐 <https://github.com/iiiiin>

▲ <https://portfolio-inkwon.vercel.app/>

경력 사항

2025.11 ~ (주)도터펫 프론트엔드 개발
2026. 01 (계약직)

학력 사항

2023.02 단국대학교 기계공학과 졸업

자격증

2024.09 정보처리기사
2023.10 SQL 개발자

수상 및 교육 사항

2025.08 공통프로젝트 우수상
(삼성 청년 SW 아카데미)

2025.01 ~ 삼성 청년 SW 아카데미 13기
2025. 10

2023.01 ~ KEPCO Digital BootCamp 1기
2023. 07

기술 스택

Frontend

 React  JavaScript  TypeScript

 Zustand  TanStack Query  TailwindCSS

Cross-platform

 Flutter  Dart  React Native  Expo

DevOps & Monitoring

 Firebase Hosting  Sentry  GitHub Actions

PROJECTS

01 Paw Print

02 RE:VIEW

03 코코의 숲

01

Paw Print

개요

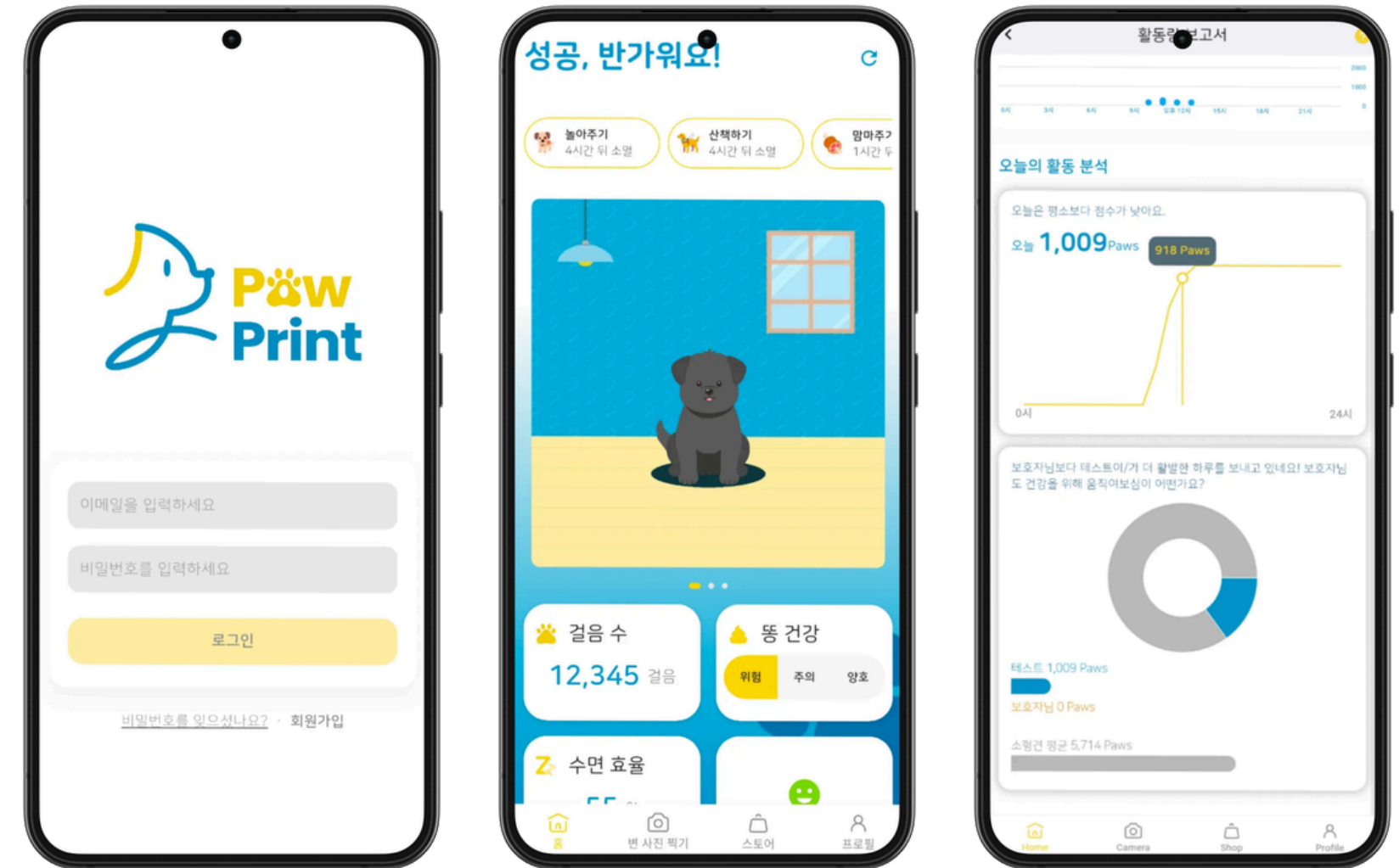
반려동물 활동량 추적 및 AI 변 이미지 건강 분석 서비스

프로젝트 정보

- 진행 기간: 2025.11 ~ 2026.01
- 참여 인원: (주)도터펫 (프론트엔드 1명, 백엔드 1명)
- 담당 역할: Flutter Web/App 페이지 API 연동, 사용자 테스트 운영

기술 스택

Flutter, RiverPod, Firebase Hosting, GitHub Actions, Sentry



01

크로스 플랫폼 보안 아키텍처 구축

문제

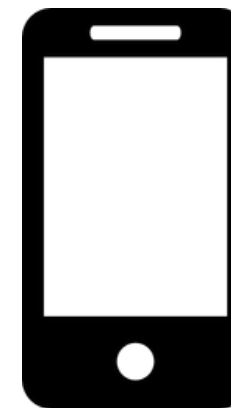
Flutter 단일 코드베이스로 웹과 앱을 동시 서비스하면서,
플랫폼별 보안 취약점(Web의 XSS 공격, App의 기기 탈취 위험)을 방어할
맞춤형 인증 전략이 필요했습니다.

해결

- **Web: HttpOnly Cookie**를 적용해 JavaScript 접근을 차단, XSS 공격을 원천 방어했습니다.
- **App: Flutter Secure Storage**를 통해 OS 레벨 암호화 저장소를 구축했습니다.
- **공통: Access Token**은 **Riverpod** 전역 상태(메모리)에만 띄워 디스크 I/O 부하를 없애고 탈취 리스크를 최소화했습니다.

결과 및 성과

하나의 코드베이스를 유지하면서도,
각 플랫폼의 보안 표준을 준수하는 JWT 인증 아키텍처를 완성했습니다.



Flutter Secure
Storage



HttpOnly Cookie
(Refresh Token)

Riverpod
(Access Token)

01

Sentry 기반 실시간 에러 트래킹 파이프라인 구축

문제

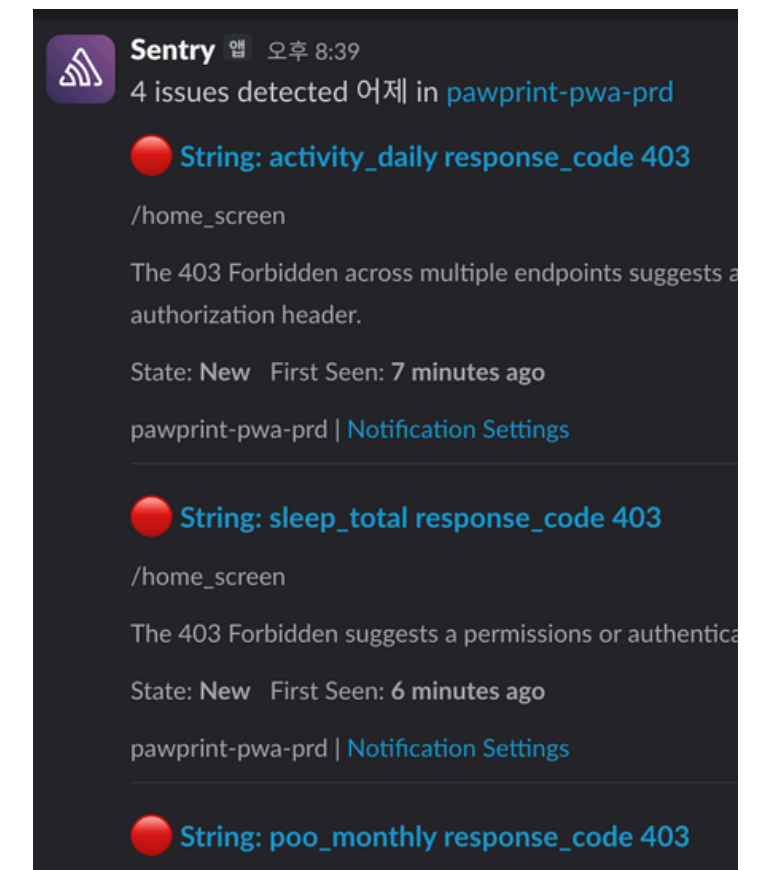
사용자 테스트 당시, 사용자의 다양한 기기 환경에서 발생하는 크리티컬 버그들을 단순 VOC 접수만으로는 재현하고 추적하는 데 한계가 있었습니다.

해결

- **에러 스택 자동 수집:** Sentry를 연동하여 앱 크래시 발생 시 콜스택(Call Stack)과 디바이스 상태 데이터를 자동으로 수집 및 시각화했습니다.
- **실시간 알림 파이프라인:** Sentry와 **Firebase Analytics**의 크리티컬 이슈를 **Slack** 웹훅과 연동하여, 개발팀이 에러를 즉시 인지할 수 있는 무중단 알림 체계를 구축했습니다.

결과 및 성과

사용자가 불편을 신고하기 전에 에러 발생을 먼저 인지하고 핫픽스(Hotfix)를 배포하는 선제적 운영 프로세스를 확립하여, 버그 원인 파악 및 대응 시간을 단축했습니다.



02

RE:VIEW

개요

AI 기반 면접 및 영상 분석 피드백 웹 서비스

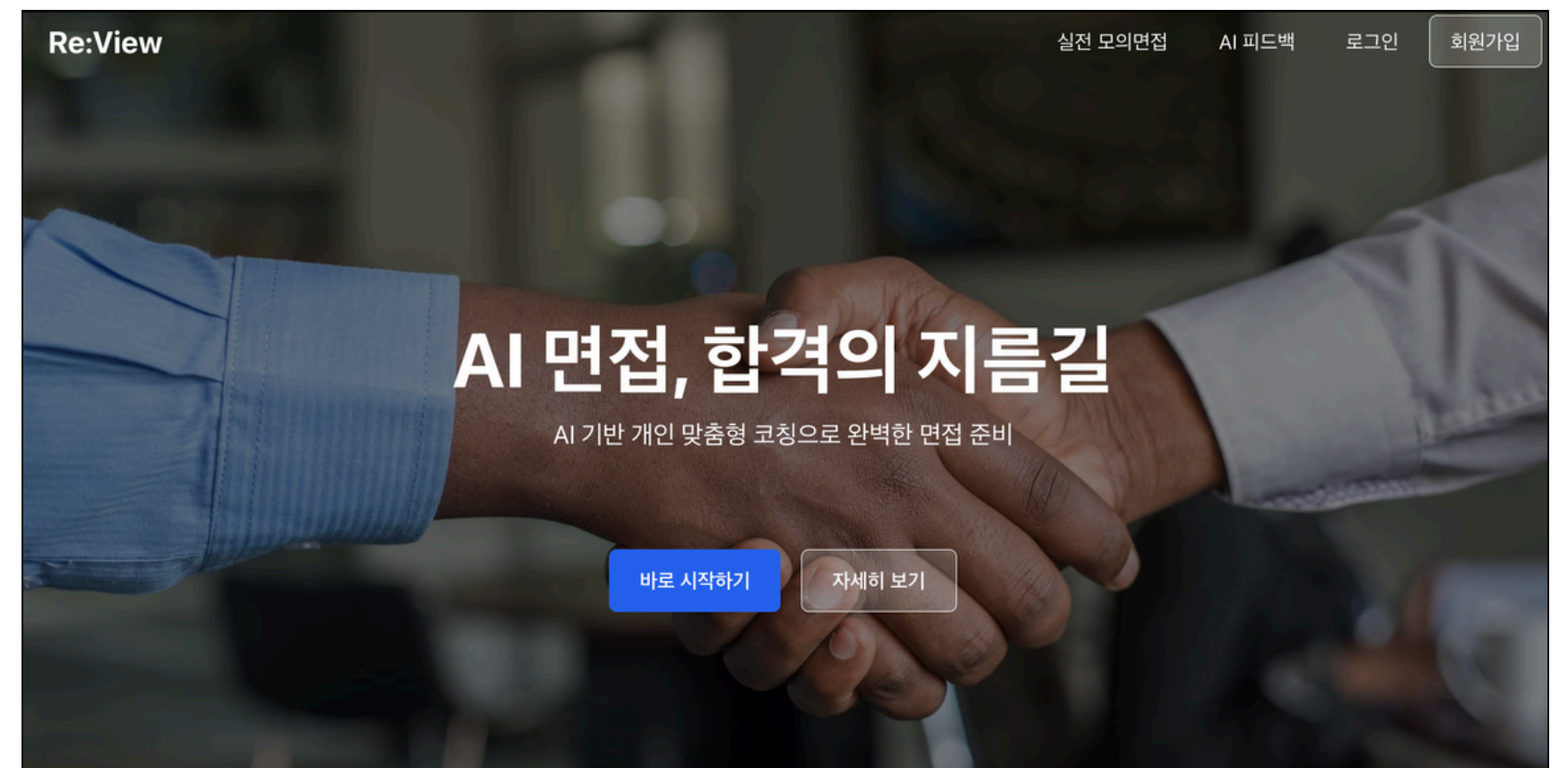
프로젝트 정보

- 진행 기간: 2025.07 ~ 2025.08 (총 1개월)
- 참여 인원: 팀 프로젝트 (프론트엔드 3명, 백엔드 2명, 인프라 1명)
- 담당 역할: React 기반 UI/UX 개발 및 API 연동

기술 스택

React, TypeScript, Zustand, TanStack Query, Tailwind CSS

 <https://github.com/iiiiin/review.git>



02

다중 API 요청 큐 직렬화 및 인증 제어

문제

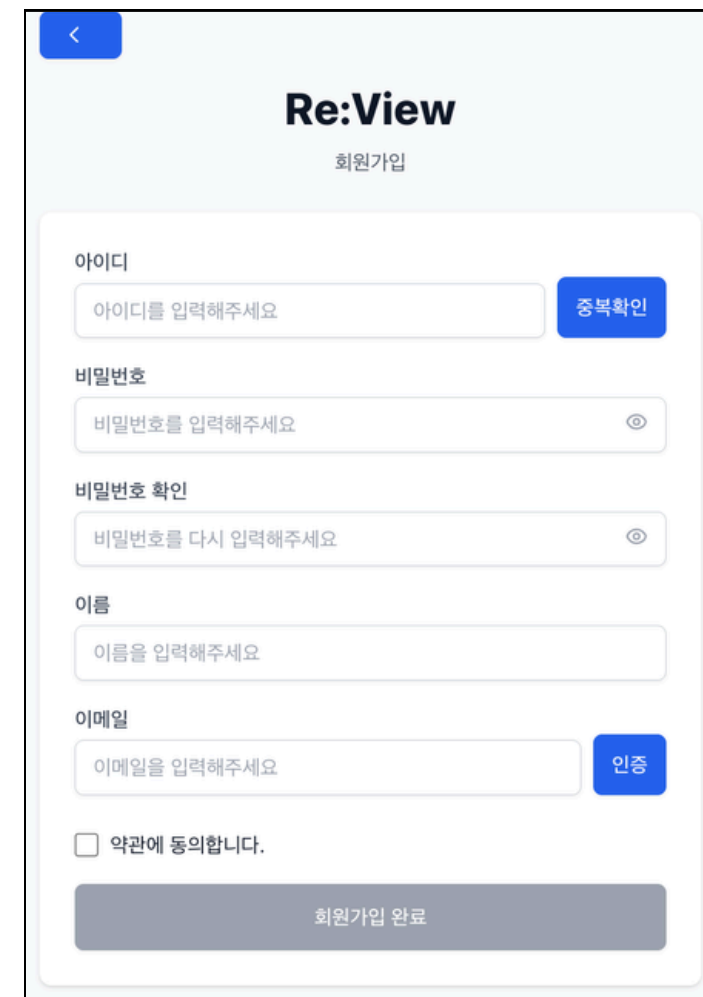
Access Token 만료 시 여러 API가 동시에 401을 반환하면, Refresh 요청이 중복 발생해 인증 실패와 레이스 조건이 생길 수 있었습니다.

해결

- **401 감지 및 요청 대기열화:** **Axios** 응답 인터셉터에서 401을 감지하고, 토큰 갱신 중(isRefreshing)에는 후속 요청을 **failedQueue**에 대기시켰습니다.
- **단일 갱신 및 재요청:** Refresh 성공 시 새 **Access Token**으로 대기 요청을 순차 재시도하고, 실패 시 공통 로그아웃/재로그인 흐름으로 정리했습니다.

결과 및 성과

토큰 만료 상황에서도 중복 Refresh 요청을 억제하고, 사용자 요청을 안전하게 복구하는 인증 복원 흐름을 구현했습니다.



Re:View
회원가입

아이디
아이디를 입력해주세요 중복확인

비밀번호
비밀번호를 입력해주세요 👁

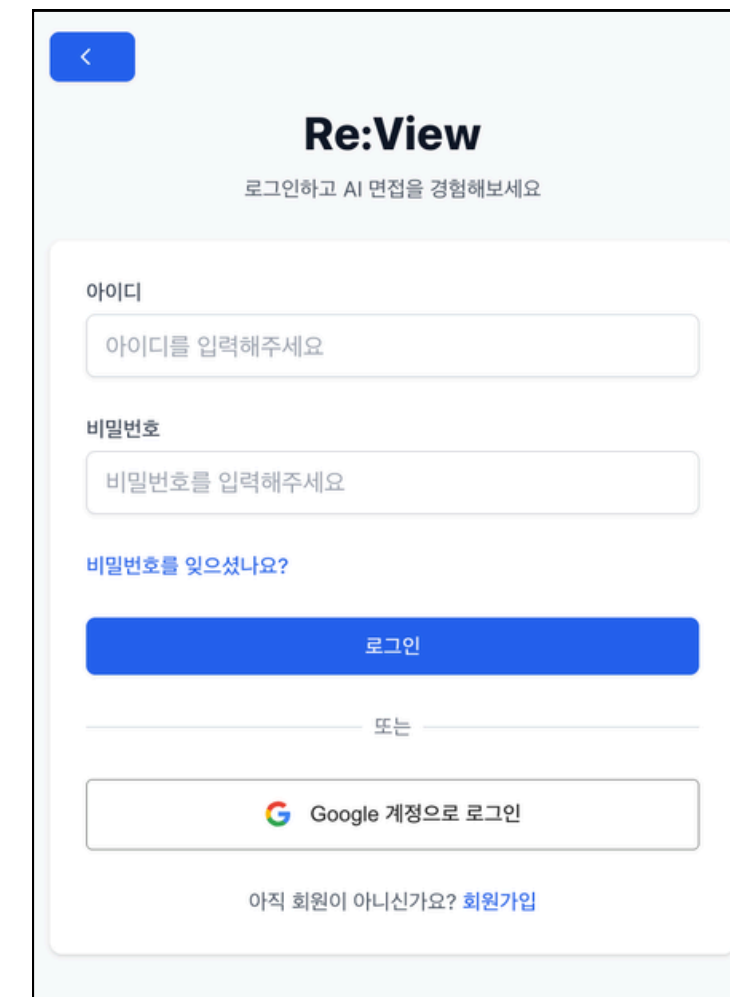
비밀번호 확인
비밀번호를 다시 입력해주세요 👁

이름
이름을 입력해주세요

이메일
이메일을 입력해주세요 인증

☐ 약관에 동의합니다.

회원가입 완료



Re:View
로그인하고 AI 면접을 경험해보세요

아이디
아이디를 입력해주세요

비밀번호
비밀번호를 입력해주세요

비밀번호를 잊으셨나요?

로그인

또는

 Google 계정으로 로그인

아직 회원이 아니신가요? [회원가입](#)

02

양방향 상태 동기화 및 결과 화면 데이터 일관성 개선

문제

결과 분석 화면에서 영상 구간과 감정 그래프를 따로 탐색해야 하는 것으로 피드백 해석에 대한 사용자 경험이 저하되었습니다.

해결

- **영상-그래프 양방향 연결:** Video currentTime과 Chart.js 감정 타임라인 포인터를 동기화하고, 차트 클릭 시 해당 시점으로 영상이 즉시 탐색되도록 구현했습니다.
- **목록 상태 동기화 전략:** 결과 목록 변경(삭제) 후에는 TanStack Query invalidateQueries로 서버 데이터와 화면 상태를 일관되게 동기화했습니다.

결과 및 성과

사용자가 영상/감정 데이터를 즉시 교차 탐색할 수 있게 되었고, 목록 변경 후 최신 상태가 안정적으로 반영되는 UX를 확보했습니다.



03

코코의 숲

개요

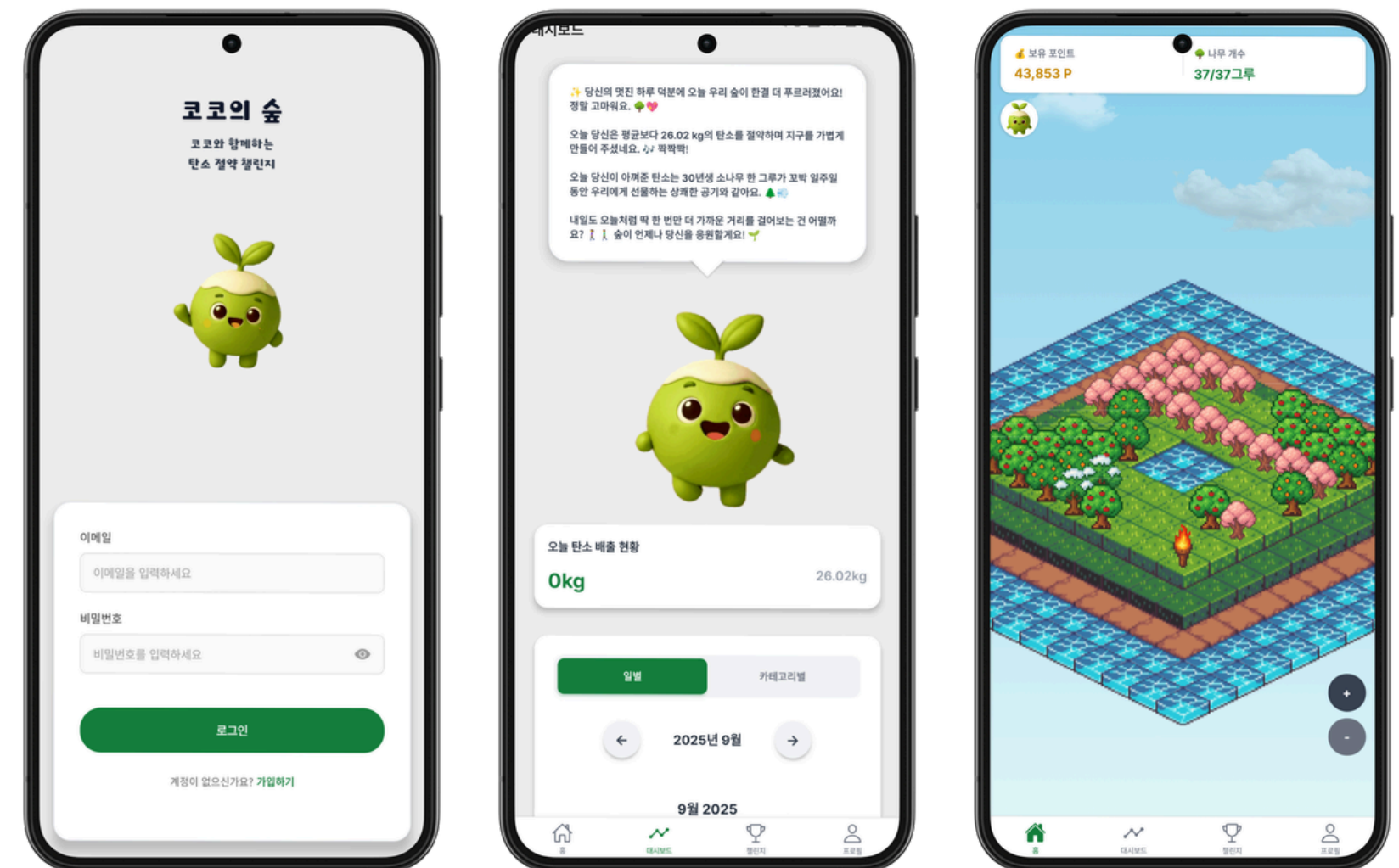
소비 내역 기반 탄소 배출량 추적 관리 앱 서비스

프로젝트 정보

- 진행 기간: 2025.09 ~ 2025.10 (총 1개월)
- 참여 인원: 팀 프로젝트 (프론트엔드 3명, 백엔드 2명, 인프라 1명)
- 담당 역할: 대시보드 API 연동, OCR 전처리 로직 구현, Expo 빌드

기술 스택

React Native, Expo, Axios, Zustand, TanStack Query



<https://github.com/iiiiin/cocos-forest.git>

03

달력 대시보드 Prefetching 및 캐싱 최적화

문제

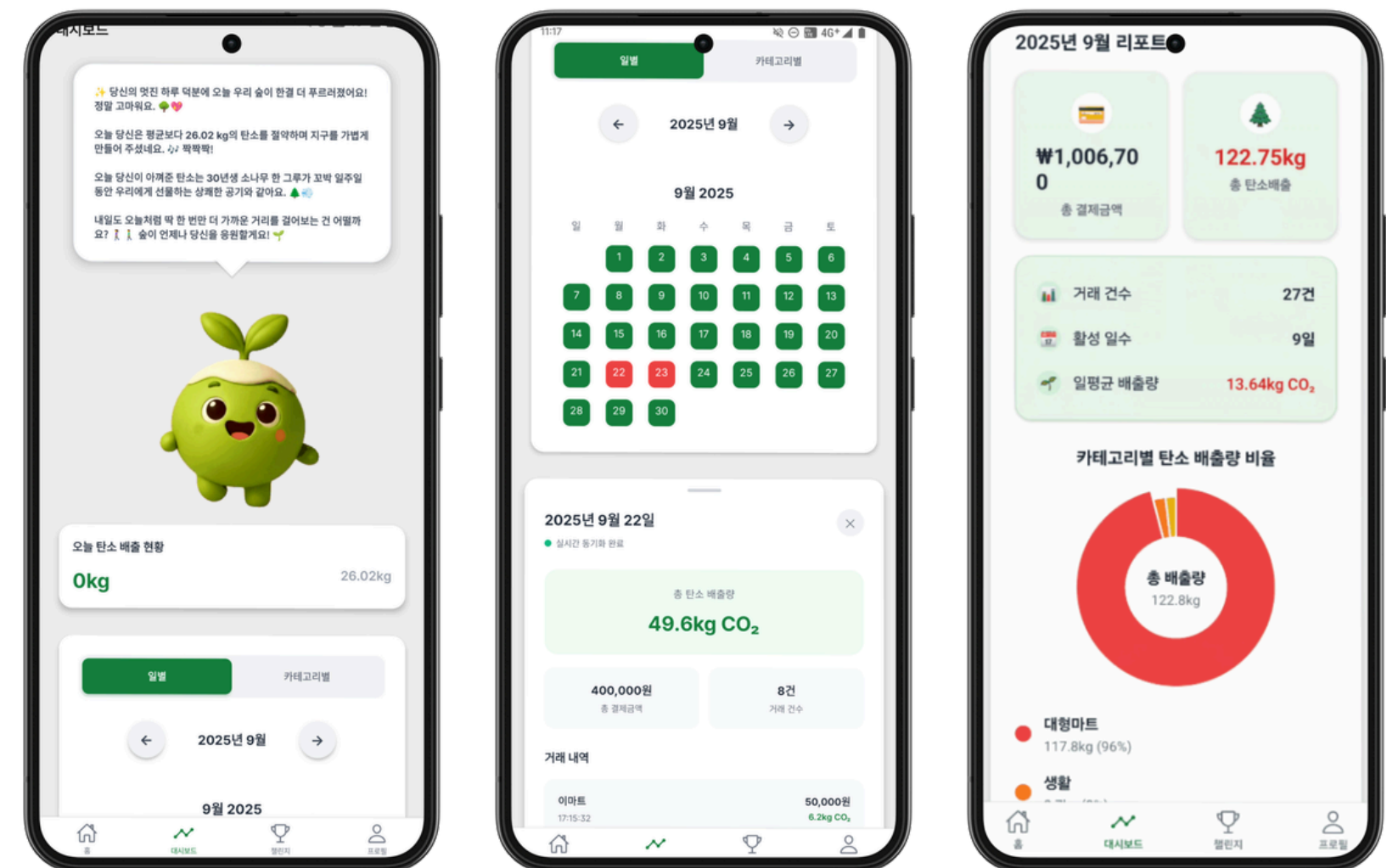
대시보드에서 사용자가 월을 이동할 때마다 월별 탄소 배출량 데이터를 새로 조회해 로딩 지연이 반복되고, 체감 탐색 속도가 저하되는 문제가 있었습니다.

해결

- **인접 월 사전 로딩(Prefetching):** `queryClient.prefetchQuery`로 이전/다음 월 리포트를 미리 캐시에 적재해, 월 전환 시 네트워크 대기 시간을 줄였습니다.
- **정적 데이터 캐싱 최적화:** 월별/일별 데이터 특성에 맞춰 `staleTime`, `gcTime`을 분리 설정하고, 기존 캐시가 신선한 경우 prefetch를 건너뛰도록 조건 분기했습니다.

결과 및 성과

월 이동 시 화면 깜빡임이나 로딩 대기 시간이 없는 부드러운 UX를 제공하였으며, 동시에 서버 네트워크 부하를 절감했습니다.



03

적응형 이미지 압축 및 업로드 최적화

문제

사용자가 영수증 이미지를 원본으로 업로드할 때 파일 크기와 네트워크 상태에 따라 업로드 실패(413)와 지연이 발생해 인증 UX가 저하되었습니다.

해결

- **적응형 압축 전략 도입:** expo-image-manipulator를 사용해 원본 파일 크기 구간별로 압축률을 동적으로 조정했습니다.
- **페이로드(Payload) 안정성 확보:** 전송되는 FormData 구성을 모바일 환경에 맞게 표준화하여 Nginx 단에서의 비정상적인 차단 이슈를 방지했습니다.

결과 및 성과

업로드 실패율을 줄이고 대용량 이미지 전송 부담을 완화해 인증 흐름 안정성과 체감 속도를 개선했습니다.



THANK YOU

권인

010-7999-5534

in24041210@gmail.com